The Ten Seed Problems of the Refute-Problem Difficulty Study

Refute User Study Team

September 27, 2026

Abstract

This document describes the ten practice problems that the study web application loads into its database when it is first set up. For each problem we give the story the student reads, the correct and the faulty program, what exactly is wrong with the faulty program, and the example inputs that are used at each step. We also explain, in plain language, what each piece of information stored with a problem is for. No programming background is needed to follow the explanations; the program listings are included for completeness.

Contents

1 What the study asks students to do **2**

2 What is stored for each problem **2**

3 Overview of the ten problems **4**

4 The problems **4**

4.1 Problem 1: `max_of_three` . . . 4
4.2 Problem 2: `smallest` . . . 4
4.3 Problem 3: `count_positives` . . . 5
4.4 Problem 4: `first_positive` . . . 5
4.5 Problem 5: `best_day` . . . 6
4.6 Problem 6: `cannot_form_teams` . . . 6
4.7 Problem 7: `runner_up` . . . 7
4.8 Problem 8: `longest_hot_streak` . . . 7
4.9 Problem 9: `max_talks` . . . 8
4.10 Problem 10: `parking_fee` . . . 8

5 How the example inputs were checked **9**

6 Order in which students see the problems **9**

1 What the study asks students to do

Every problem in the study is a *refute problem*: the student is given a description of what a small program should do, together with a version of that program that contains a mistake. The student's ultimate goal is to find a *counter-example* — an input on which the faulty program gives a different answer from a correct one.

To see *where* students struggle, the task is split into three steps, each timed separately:

Step 1 — Understanding the task.
The student sees only the description (no program) and writes three example inputs with the answers they expect. The system silently checks these against the correct program. This tells us whether the student understood what the program is *supposed* to do.

Step 2 — Reading the faulty program.
The student now sees the faulty program and answers questions about how it behaves on inputs chosen by the researchers: how many times each line runs, and what values the program's variables hold as it runs. The chosen inputs never reveal the mistake, so this measures pure reading skill. There is one attempt and no feedback.

Step 3 — Finding the counter-example.
The student proposes an input and predicts what both the correct and the faulty program would answer. The step is complete when the input really does make the two programs disagree *and* both predictions are right.

2 What is stored for each problem

Each problem is stored as one record with the fields below. The names in typewriter font are the exact names used in the file `backend/seed_data.py`.

Field	What it means, in plain terms
`title`	A short, unique name for the problem. It is also used to recognise a problem that has already been loaded, so loading twice never creates duplicates.
`language`	The programming language (currently always Python). The home page only offers problems in the language the student picks.
`description`	The task as the student reads it in Step 1 — either a direct statement ("return the smallest number") or a real-world story that the student must translate into a computation.
`function_signature`	The first line of the program (its name and inputs), shown as a heading above the Step 1 answer form.
`correct_code`	A correct program for the task. Students never see it; it is the hidden "answer key" their Step 1 examples and Step 3 predictions are checked against.
`buggy_code`	The faulty program the student studies in Steps 2 and 3. It differs from the correct one in a small, deliberate way.
`trace_sample_inputs`	The input(s) used for the "how many times does each line run?" question in Step 2. They are chosen so that the mistake does *not* show up on them.
`trace_data_flow_inputs`	A *different* input used for the "what value does each variable hold?" question in Step 2. Here the order in which lines run is shown, and only the values are asked. It must differ from the input above, otherwise showing the order would give away the answer to the first question.
`buggy_line_numbers`	Which line(s) of the faulty program contain the mistake. Used only in the analysis — for example, to ask "did the student read the faulty line correctly?" — and never shown to students.
`trace_prefill_steps`	How many rows of each Step 2 table are filled in already, as a worked example of how to answer (1 for every problem).
`trace_omit_unchanged_vars`	Whether to hide variables whose value never changes from the value table. Always off here, so students are asked about every variable.
`io_example_inputs`	Two example inputs whose correct answers are shown to a student who gets stuck in Step 1 (only after a wrong attempt). The student may not then reuse these as their own examples.
`mutation_line_number`, `mutation_new_line_text`, `mutation_prompt`	An optional reflection question at the end of Step 2. Either a line is shown changed to new text and the student describes what difference that makes, or (when no new text is given) the student answers a question about the program as it stands. Answers are free text and are not marked.

Every problem is chosen to be *small* (a single short program, no program that calls itself, at most a few dozen steps to trace) and to return a number or true/false, so that students can type answers without worrying about formatting.

3 Overview of the ten problems

#	Title	Kind of mistake	Style
1	`max_of_three`	an "else-if" that skips a needed check	direct
2	`smallest`	the loop stops one item too early	direct
3	`count_positives`	"at least zero" instead of "above zero"	direct
4	`first_positive`	keeps looking after finding the answer	direct
5	`best_day`	compares with the wrong earlier value	story
6	`cannot_form_teams`	the search stops too soon (fails only for 4)	story
7	`runner_up`	two updates done in the wrong order	story
8	`longest_hot_streak`	forgets to count a streak still running at the end	story
9	`max_talks`	remembers when a talk starts, not when it ends	story
10	`parking_fee`	rounds up wrongly on exact whole hours	story

Problems 1–4 state the computation directly. Problems 5–10 are *indirect*: the description is an everyday situation, and part of the challenge is recognising the computation hidden in the story.

4 The problems

In the code listings, line numbers are shown on the left; the "faulty line" mentioned in each summary refers to these numbers in the faulty program.

4.1 Problem 1: `max_of_three`

Description. Consider a function `max_of_three(a, b, c)` that takes three integers and returns the largest of the three.

Correct

```
1 def max_of_three(a, b, c):
2     largest = a
3     if b > largest:
4         largest = b
5     if c > largest:
6         largest = c
7     return largest
```

Faulty

```
1 def max_of_three(a, b, c):
2     largest = a
3     if b > largest:
4         largest = b
5     elif c > largest:
6         largest = c
7     return largest
```

The bug: Line 5 says "otherwise, if" (`elif`) instead of "if". Once `b` has beaten `a`, the program never looks at `c`, so it misses a `c` that is larger still. Counter-example: `max_of_three(1, 3, 5)` should give 5, but the faulty program gives 3.

- Step 2 inputs: `max_of_three(2, 1, 3)` (line counts) and `max_of_three(1, 5, 3)` (values).
- Help examples: `max_of_three(4, 9, 2)`, `max_of_three(-3, -1, -7)`.
- Reflection: line 5 shown changed back to `if c > largest:` (the actual fix); "describe how the behaviour changes".

4.2 Problem 2: `smallest`

Description. Write a function `smallest(nums)` that returns the smallest number in the list `nums`. You may assume `nums` contains at least one number.

Correct

```
1 def smallest(nums):
2     result = nums[0]
3     for i in range(1, len(nums)):
4         if nums[i] < result:
5             result = nums[i]
6     return result
```

Faulty

```
1 def smallest(nums):
2     result = nums[0]
3     for i in range(1, len(nums) - 1):
4         if nums[i] < result:
5             result = nums[i]
6     return result
```

The bug: Line 3 stops the loop one item early, so the last number in the list is never looked at. Counter-example: `smallest([10, 20, 5])` should give 5, but the faulty program gives 10.

- Step 2 inputs: `smallest([5, 3, 8])` and `smallest([9, 2, 6, 4])`.
- Help examples: `smallest([10, 20, 5])`, `smallest([-3, -7, -1])`.
- Reflection: line 4 shown with "<" flipped to ">"; "describe the resulting code in one line".

4.3 Problem 3: `count_positives`

Description. Write a function `count_positives(nums)` that returns how many numbers in the list `nums` are strictly greater than zero.

Correct

```
1 def count_positives(nums):
2     total = 0
3     for x in nums:
4         if x > 0:
5             total = total + 1
6     return total
```

Faulty

```
1 def count_positives(nums):
2     total = 0
3     for x in nums:
4         if x >= 0:
5             total = total + 1
6     return total
```

The bug: Line 4 counts numbers that are *at least* zero, so zero is wrongly counted as positive. Counter-example: `count_positives([0, 1, 2])` should give 2, but the faulty program gives 3.

- Step 2 inputs: `count_positives([1, -2, 3, -4])` and `count_positives([-5, -6, -7])` (no zeros, so the mistake stays hidden).
- Help examples: `count_positives([2, -1, 5])`, `count_positives([-10, -20])`.
- Reflection: line 5 shown changed to `total = total + x` (adding up instead of counting); "describe the resulting code".

4.4 Problem 4: `first_positive`

Description. Write a function `first_positive(nums)` that returns the first element of the list `nums` that is strictly greater than zero. If no such element exists, return `None`.

Correct

```
1 def first_positive(nums):
2     for x in nums:
3         if x > 0:
4             return x
5     return None
```

Faulty

```
1 def first_positive(nums):
2     result = None
3     for x in nums:
4         if x > 0:
5             result = x
6     return result
```

The bug: The faulty program never stops once it finds a positive number; each later positive number overwrites the earlier one (line 5), so it returns the *last* positive number, not the first. Counter-example: `first_positive([2, -1, 7])` should give 2, but the faulty program gives 7.

- Step 2 inputs: `first_positive([-1, -2, 5, -3])` and `first_positive([-5, -6, 8])` (only one positive number each, so "first" and "last" coincide).
- Help examples: `first_positive([-1, -2, -3])`, `first_positive([4, -5, -6])`.

- Reflection (no changed line): "If `nums` has 10 elements, how many times will line 5 be executed?"

4.5 Problem 5: `best_day`

Description. A shop records its sales for a number of consecutive days in the list `sales`, where `sales[0]` is day 0, `sales[1]` is day 1, and so on. Write a function `best_day(sales)` that returns the day on which the shop sold the most. If several days tie for the most sales, return the earliest of them. You may assume the shop was open for at least one day.

Hidden computation. The position of the largest number in a list.

Correct

```
1 def best_day(sales):
2     best = 0
3     for d in range(1, len(sales)):
4         if sales[d] > sales[best]:
5             best = d
6     return best
```

Faulty

```
1 def best_day(sales):
2     best = 0
3     for d in range(1, len(sales)):
4         if sales[d] > sales[d - 1]:
5             best = d
6     return best
```

The bug: Line 4 compares each day with the *previous* day instead of with the best day so far, so it reports the last day that beat the day before it. Counter-example: `best_day([5, 1, 3])` should give day 0, but the faulty program gives day 2.

- Step 2 inputs: `best_day([7, 2, 1])` and `best_day([2, 8, 3])`.
- Help examples: `best_day([4, 9, 6])`, `best_day([5, 5, 2])` (the second shows the tie rule).
- Reflection: line 4 shown with the fix, `if sales[d] > sales[best]:`; "describe how the behaviour changes".

4.6 Problem 6: `cannot_form_teams`

Description. A coach has `n` players (`n` is at least 2) and wants to split all of them into teams of equal size, with at least 2 players per team and at least 2 teams. Write a function `cannot_form_teams(n)` that returns `True` if this is impossible and `False` if it can be done.

Hidden computation. Whether `n` is a prime number (a split is possible exactly when `n` can be written as a product of two numbers that are each at least 2).

Correct

```
1 def cannot_form_teams(n):
2     for size in range(2, n):
3         if n % size == 0:
4             return False
5     return True
```

Faulty

```
1 def cannot_form_teams(n):
2     for size in range(2, n // 2):
3         if n % size == 0:
4             return False
5     return True
```

The bug: Line 2 tries too few team sizes. For almost every `n` this still finds a valid split, but for `n = 4` it tries no sizes at all and wrongly says a split is impossible. **4 is the only counter-example** (checked for every `n` from 2 to 100), which makes this deliberately the hardest Step 3 in the set.

- Step 2 inputs: `cannot_form_teams(15)` and `cannot_form_teams(7)`.
- Help examples: `cannot_form_teams(11)`, `cannot_form_teams(9)`.
- Reflection (no changed line): "If `n` is 20, how many times will line 3 be executed?" (20 is used rather than 4 so the question does not give away the counter-example.)

4.7 Problem 7: `runner_up`

Description. In a contest, every participant gets a different score, recorded in the list `scores` in no particular order. Write a function `runner_up(scores)` that returns the score of the participant who came second. You may assume there are at least two participants.

Hidden computation. The second-largest number in a list.

Correct

```
1  def runner_up(scores):
2      first = max(scores[0], scores[1])
3      second = min(scores[0], scores[1])
4      for s in scores[2:]:
5          if s > first:
6              second = first
7              first = s
8          elif s > second:
9              second = s
10     return second
```

Faulty

```
1  def runner_up(scores):
2      first = max(scores[0], scores[1])
3      second = min(scores[0], scores[1])
4      for s in scores[2:]:
5          if s > first:
6              first = s
7              second = first
8          elif s > second:
9              second = s
10     return second
```

The bug: Lines 6 and 7 are in the wrong order: the old winner should become second *before* the new winner is recorded. As written, "second" copies the *new* winner, so first and second end up equal. Counter-example: `runner_up([1, 2, 3])` should give 2, but the faulty program gives 3. Any list where the top score appears after the first two entries exposes it.

- Step 2 inputs: `runner_up([9, 5, 7])` and `runner_up([4, 6, 1, 5])` (the top score is among the first two, so lines 6–7 never run).
- Help examples: `runner_up([3, 8])`, `runner_up([10, 2, 7])`.
- Reflection (no changed line): "If there are 6 participants, what is the largest number of times line 9 can be executed?"

4.8 Problem 8: `longest_hot_streak`

Description. A weather station records one temperature per day in the list `temps`. Write a function `longest_hot_streak(temps)` that returns the length of the longest run of consecutive days on which the temperature was above 30, or 0 if there were no such days.

Hidden computation. The longest run of consecutive list items above a threshold.

Correct

```
1  def longest_hot_streak(temps):
2      best = 0
3      run = 0
4      for t in temps:
5          if t > 30:
6              run = run + 1
7              if run > best:
8                  best = run
9          if t <= 30:
10             run = 0
11     return best
```

Faulty

```
1  def longest_hot_streak(temps):
2      best = 0
3      run = 0
4      for t in temps:
5          if t > 30:
6              run = run + 1
7          if t <= 30:
8              if run > best:
9                  best = run
10             run = 0
11     return best
```

The bug: The record is only updated when a streak *ends* on a cooler day (lines 8–9). A hot streak that is still going on the last day is never counted. Counter-example: `longest_hot_streak([31, 32])` should give 2, but the faulty program gives 0.

- Step 2 inputs: `longest_hot_streak([31, 25, 33, 34, 20])` and `longest_hot_streak([35, 28, 29])` (both end on a cool day, so every streak gets closed and counted).
- Help examples: `longest_hot_streak([25, 20])`, `longest_hot_streak([33, 31, 22, 35])`.
- Reflection (no changed line): "In one sentence, what does `best` hold after the loop ends?"

4.9 Problem 9: `max_talks`

Description. A conference lists its talks as `(start, end)` pairs in the list `talks`, sorted by end time. Going through the list in order, you attend a talk whenever it starts no earlier than the moment your previous talk ended (back-to-back talks are fine). Write a function `max_talks(talks)` that returns how many talks you attend.

Hidden computation. A "greedy" selection of non-overlapping time slots.

Correct

```
1 def max_talks(talks):
2     count = 0
3     free_at = 0
4     for start, end in talks:
5         if start >= free_at:
6             count = count + 1
7             free_at = end
8     return count
```

Faulty

```
1 def max_talks(talks):
2     count = 0
3     free_at = 0
4     for start, end in talks:
5         if start >= free_at:
6             count = count + 1
7             free_at = start
8     return count
```

The bug: Line 7 remembers when the attended talk *started* rather than when it *ends*, so a later talk that overlaps it is wrongly accepted. Counter-example: `max_talks([(1, 4), (2, 5)])` should give 1, but the faulty program gives 2.

- Step 2 inputs: `max_talks([(1, 2), (3, 5), (6, 8)])` and `max_talks([(2, 3), (4, 6)])` (no overlaps).
- Help examples: `max_talks([(1, 3), (3, 4)])` (shows back-to-back is allowed), `max_talks([(1, 10)])`.
- Reflection: line 7 shown with the fix, `free_at = end`; "describe how the behaviour changes".

4.10 Problem 10: `parking_fee`

Description. A car park charges 20 for the first hour and 10 for each additional hour. Any part of an hour counts as a full hour. The fee for a single stay never exceeds 100. Write a function `parking_fee(minutes)` that returns the fee for parking `minutes` minutes. You may assume `minutes` is at least 1.

Hidden computation. Rounding up to whole hours, then a fee with a maximum.

Correct

```
1 def parking_fee(minutes):
2     hours = (minutes + 59) // 60
3     fee = 20 + (hours - 1) * 10
4     if fee > 100:
5         fee = 100
6     return fee
```

Faulty

```
1 def parking_fee(minutes):
2     hours = minutes // 60 + 1
3     fee = 20 + (hours - 1) * 10
4     if fee > 100:
5         fee = 100
6     return fee
```

The bug: Line 2 rounds up correctly *except* for stays of an exact number of hours, where it adds an extra hour that was never used. Counter-example: `parking_fee(60)` should give 20,

but the faulty program gives 30. The only counter-examples are 60, 120, ..., 480 minutes: from 540 minutes upward both programs hit the maximum of 100, which hides the mistake.

- Step 2 inputs: `parking_fee(90)` and `parking_fee(200)` (neither is a whole number of hours).
- Help examples: `parking_fee(30)`, `parking_fee(1000)` (the second shows the maximum).
- Reflection: line 2 shown with the fix, `hours = (minutes + 59) // 60`; "describe how the behaviour changes".

5 How the example inputs were checked

For every problem, each Step 2 input and each help example was run through both programs to confirm that they give the *same* answer (so none of them reveals the mistake), and each counter-example quoted above was run to confirm that the answers *differ*. For `cannot_form_teams` and `parking_fee`, where the claim is that only a few specific inputs expose the mistake, every input in the relevant range was tried (2–100 players; 1–1440 minutes) to confirm the claim.

6 Order in which students see the problems

Problems are presented in the order listed above (by their number in the database), and a student who picks Python works through all ten. The order is not currently shuffled between students; if the study design needs to rule out order effects, that remains to be added.